Games Engineering Report

Linlang Zou
5684282

University of Warwick

February 15, 2026

This report describes the rasteriser optimisations and multithreading done for Part 1, and the chat room for Part 2.

Part I

Part 1: Optimising a Rasteriser

1 Introduction (Part 1)

I optimised the provided base rasteriser and then added multithreading with C++ `std::thread`. Output is unchanged: same geometry, lighting and image for the same inputs.

What I did: (1) Matrix and vector maths in `matrix.h` and `vec4.h` were sped up with SSE4.1 SIMD and 16-byte alignment. (2) Vertex cache: in the multithreaded path each vertex is only transformed once per frame and then reused for all triangles that use it. (3) In `triangle.h` I added an AVX2 path that does 8 pixels at a time for the inner loop. (4) For multithreading I split the screen into 256×256 tiles and use a thread pool so each tile is one job; triangles are binned to the tiles they touch. I set the thread count to 11 to match my machine.

I tested on the two given scenes and my own Scene 3 (six spheres in orbit plus a 10×10 grid of spheres). All timings are in Release at 1024×768. **Speedup is reported by optimisation stage, not by thread count**: five stages from the original (Stage 0) to adding in turn (1) Transformation (Matrix and Vector), (2) Vertex Transform Reuse (Vertex Cache), (3) Rasterisation Optimisations (Triangle SIMD), (4) multithreading. The setup I used is shown in Figure 1.

Figure 1: *Setup for all Part 1 timings.*

2 Optimisations

2.1 Transformation (Matrix and Vector)

Each vertex does a matrix times vector and the normal transform, so I tried to speed that up with SIMD. In `matrix.h` I added `alignas(16)` and a union with `__m128 row[4]`. The matrix times vec4 (lines 38–60) now uses `_mm_dp_ps` for each row; matrix times matrix (66–94) uses the SSE instructions per row. In `vec4.h` I added `alignas(16)` so the loads are aligned.

```
1 __m128 vec = _mm_loadu_ps(v.getV());
2 __m128 row0 = _mm_loadu_ps(&a[0]);
3 __m128 res0 = _mm_dp_ps(row0, vec, 0xF1);
4 __m128 res1 = _mm_dp_ps(row1, vec, 0xF1);
5 result[0] = _mm_cvtss_f32(res0);
6 result[1] = _mm_cvtss_f32(res1);
```

Listing 1: *Matrix×vec4 SIMD (matrix.h).*

2.2 Vertex Transform Reuse (Vertex Cache)

In the base code every triangle transforms its three vertices in `render()`, so shared vertices get done many times. With the vertex cache path,'`fillFrameVertexCaches()` (33–55) builds one cache per mesh and `buildAllTrianglesFromCache()` (58–75) builds the triangle list from it, so each vertex is only transformed once per frame. Key code (lines 39–52):

```
1  matrix p = renderer.perspective * camera * mesh
       ->world;
2  for (size_t vi = 0; vi < mesh->vertices.size();
       vi++) {
3      Vertex& out = cache[vi];
4      out.p = p * mesh->vertices[vi].p;
5      out.p.divideW();
6      out.normal = mesh->world * mesh->vertices[vi
       ].normal;
7      out.normal.normalise();
8      out.p[0] = (out.p[0] + 1.f) * 0.5f * w;
9      out.p[1] = (out.p[1] + 1.f) * 0.5f * h;
10     out.p[1] = h - out.p[1];
11     out.rgb = mesh->vertices[vi].rgb;
12 }
```

Listing 2: *Single transform per vertex (raster.cpp, lines 39–52).*

2.3 Rasterisation Optimisations (Triangle SIMD)

The inner loop does barycentrics, depth, normals, colour and lighting per pixel. I added an AVX2 path in `triangle.h`: `getC_SIMD()` and `interpolate_SIMD()` do 8 pixels at once with `__m256`, and `drawTileSIMD()` (126–204) does the 8-wide loop. `draw()` and `drawParallel()` use it for most of the row and fall back to scalar for the rest.

```
1  for (int x = xStart; x < xEnd; x += 8) {
2      __m256 px = _mm256_setr_ps((float)x, (float)
       (x+1), ..., (float)(x+7));
3      __m256 py = _mm256_set1_ps((float)y);
4      __m256 alpha = getC_SIMD(v0x, v0y, v1x, v1y,
        px, py);
5      __m256 beta  = getC_SIMD(v1x, v1y, v2x, v2y,
        px, py);
6      alpha = _mm256_mul_ps(alpha, area_inv);
7      beta  = _mm256_mul_ps(beta, area_inv);
8      __m256 gamma = _mm256_sub_ps(_mm256_set1_ps
       (1.0f), _mm256_add_ps(alpha, beta));
9      __m256 depth = interpolate_SIMD(beta, gamma,
        alpha, p0z, p1z, p2z);
```

Listing 3: *8-pixel barycentrics and depth (triangle.h, lines 134–157).*

3 Multithreading Strategy and Performance

3.1 Overview and Choice of Strategy

- Multithreading is implemented with the C++ `std::thread` library (via a custom `ThreadPool` in `Rasterizer/ThreadPool.h`).
- Strategy: **tile-based parallelism**. The screen is divided into tiles (e.g. 256×256); each tile is one job. Workers pull jobs from a shared queue; each job draws only the pixels in its tile.
- Thread count: `MT_POOL_SIZE = 11` in `raster.cpp` matches my machine's logical cores.

3.2 Task Breakdown

- **Main thread (per frame):** (1) Transform all vertices once per mesh (`fillFrameVertexCaches`). (2) Build triangle list from caches (`buildAllTrianglesFromCache`). (3) For each tile, determine which triangles overlap it (`buildTrianglesPerTile` — triangle–tile binning). (4) Submit one job per tile to the thread pool; each job runs `runTileJobFromContext(tileIndex)`. (5) `waitIdle()` before presenting.
- **Worker threads:** Each job draws only the triangles that overlap its tile, and only within the tile's pixel bounds `[startX,endX)` × `[startY,endY)`. Implemented in `drawParallel()` in `triangle.h` (with `USE_THREAD_TILE` and optional `USE_SIMD`).

The tile job and batch submission are implemented as follows (see `raster.cpp`, lines 114–124 and 159–162). `runTileJobFromContext()` computes `startX, startY, endX, endY` from the tile index, then iterates over `(*ctx->perTile)[tileIndex]` and calls `drawParallel()` for each triangle. `submitBatch(jobs)` and `waitIdle()` ensure all tiles are finished before the next frame.

```
1 const int startX = tx * MT_TILE_W;
2 const int startY = ty * MT_TILE_H;
3 const int endX = (std::min)(startX + MT_TILE_W,
      ctx->w);
4 const int endY = (std::min)(startY + MT_TILE_H,
      ctx->h);
5 const std::vector<size_t>& list = (*ctx->perTile
      )[tileIndex];
6 for (size_t i : list)
7     allTriangles[i].drawParallel(*ctx->r, *ctx->
      L, startX, startY, endX, endY);
```

Listing 4: *Tile job (raster.cpp, lines 114–124).*

Tiles are disjoint (no pixel or Z-buffer written by more than one thread), so no per-pixel locks. Vertex caches, triangle list, and per-tile indices are written only by the main thread before `submitBatch`; workers

read them. `waitIdle()` ensures the frame is finished before the next one starts.

3.3 Scene Testing and Choice of Scene 3

Scene 1 and Scene 2 are the ones that were given (Scene 1: 40 cubes in two columns with random rotations; Scene 2: 6×8 grid of cubes plus a moving sphere). For Scene 3,I did six spheres orbiting in the XY plane (radius 2.2) plus a 10×10 grid of static spheres further back. That way there is a mix of moving and static stuff so the tiles get different amounts of work and you can see the benefit of the vertex cache and the tile parallelism. All done with `Mesh::makeSphere`.

How the millisecond output works. In all three scenes the program runs a continuous render loop and prints a line each time a fixed "segment" of the animation completes. For Scene 1 the segment is one full back-and-forth of the camera (two reversals of direction); for Scene 2 it is one full back-and-forth of the moving sphere; for Scene 3 it is two complete orbits of the six spheres. So the program keeps outputting lines `1 : ...,2 : ...,3 : ...` as the animation runs; each value is the wall-clock time in milliseconds for that segment. Because the animation step per frame is fixed, segment duration is proportional to average frame time, so these values are suitable for comparing different optimisation stages.

How I get T_k and the speedup. I measure **five stages**: Stage 0 = original (no optimisations); Stage 1 = +Transformation (Matrix and Vector); Stage 2 = +Vertex Transform Reuse (Vertex Cache); Stage 3 = +Rasterisation Optimisations (Triangle SIMD); Stage 4 = +multithreading. For each scene and each stage $k = 0, 1, \ldots, 4$ I run once with that configuration and take the **first 10** printed millisecond values, then use their **average** as T_k. Speedup at stage k is T_0/T_k (baseline time divided by time at that stage). I plot **speedup vs. optimisation stage**, not vs. thread count. Using the first 10 outputs smooths out early-frame variance (e.g. cache warm-up) while keeping the procedure the same for every run.

4 Conclusion (Part 1)

I sped up the base rasteriser with matrix/vector SIMD, vertex cache, optional 8-wide triangle SIMD, and tile-based multithreading. The output is the same as the base. I measured segment times in Release at 1024×768 across five optimisation stages (original, then adding each of the four optimisations in turn) and got a clear speedup on all three scenes; the graph is below.

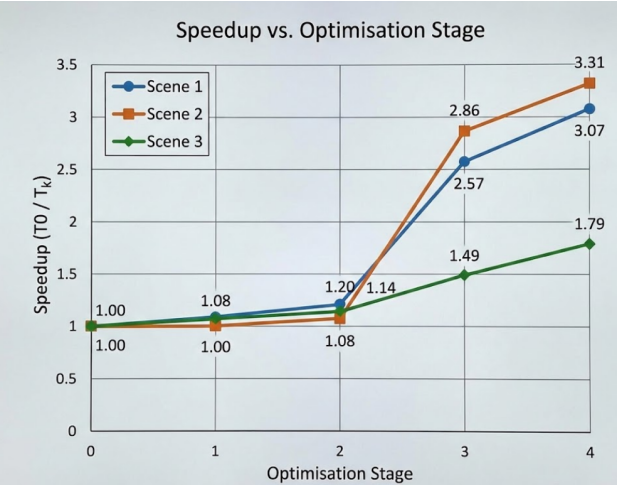

Figure 2: *Speedup vs. optimisation stage for Scene 1, Scene 2 and Scene 3. Stage 0 = original; 1 = +Transformation (Matrix and Vector); 2 = +Vertex Cache; 3 = +Rasterisation Optimisations (Triangle SIMD); 4 = +multithreading.*

Limitations: the tile size and thread count are fixed (need to recompile to change), and when using triangle SIMD there is still a scalar loop for the last few pixels of each row. There are no locks on the frame buffer or Z-buffer because each tile only writes its own pixels.

If I had more time I would make tile size and thread count configurable, try other ways to split the work (e.g. per mesh), and maybe add more SIMD or a different memory layout.

Speedup results. As above, T_k is the average of the first 10 printed segment times (ms) at stage k; speedup is T_0/T_k. The graph below plots **speedup vs. optimisation stage** (0 = original, 1 = +Transformation (Matrix and Vector), 2 = +Vertex Cache, 3 = +Rasterisation Optimisations (Triangle SIMD), 4 = +multithreading) for all three scenes in one figure.

Part II
Part 2: Developing a Chat Room

5 Introduction (Part 2)

For Part 2 I built a chat room with two programs: a server (myServer) and a client (myClient). Networking is done with WinSock and the client UI with ImGui. The server can have several clients connected at once. It forwards public (broadcast) messages to everyone and private ones only to the right person, and it handles people joining and leaving. On the client side there is a login screen, then a main window with the

user list on the left and the public chat in the middle. Direct messages open in separate windows. When you get a message it plays a sound; I used two different sounds so you can tell public from private. I used C++ `std::thread` for the server's accept loop and for each client's handler, and for the client's receive thread.

6 Client-Server Functionality

6.1 Architecture

The server listens on port 65432. When it starts, a thread runs `accept()` in a loop. For each new connection the server first reads one message — that's the username — and adds the client to a list (`clients_`, with a mutex). It sends this client the list of who's online and also broadcasts that list to everybody else so they see the new user. Then it spawns another thread (`handleClient`) that just reads from that socket in a loop. On the client, when you type your name and click Connect, a background thread does the WSAStartup, creates the socket, connects, sends the username, and then starts a receive thread. The receive thread puts messages into a queue. The GUI thread (main thread) checks the queue every frame with `hasNewMessages()` / `getNewMessages()` and updates the display.

6.2 Message Format and Routing

Messages are plain strings. I use a type prefix: everything before the first / is the type, the rest is the payload.

- On connect the client just sends the username (no prefix). The server keeps it and uses it when building the user list and for DMs.
- The user list is `clients/` plus a comma-separated list, e.g. `clients/Alice,Bob,`. The server sends this to the new client and also broadcasts it whenever someone joins or leaves.
- For public chat the client sends `public/`*username*`:` *content*. In `parseMessage()` on the server, if the type is `public` it calls `broadcastMessage` so every other client gets the same string. So everyone sees it in the main room.
- For a DM the client sends `private/`*targetName*`:` *content*. The server looks up *targetName* in `clients_` and sends only to that socket with `sendToClient`. So only that person gets the message.
- To disconnect the client sends `!bye`. The server's `handleClient` then leaves the loop, removes the client from the list, broadcasts the new `clients/` list, and closes the socket. On the client, `disconnect()` sends `!bye`, stops the receive thread, then closes the socket and does WSACleanup.

I didn't add accounts or save chat history — once you're connected you're in for that session.

7 GUI Design and Interaction

The client uses ImGui with Win32 and DirectX 12. The layout is basically three bits: login, main chat, and the DM windows.

7.1 Login Screen

First you get a full-screen "Login" window. You type your username and click "Confirm" — that starts `initializeAsync(username)` in a background thread so the UI doesn't freeze. When the connection is done (or fails) you click "Connect" to actually go into the chat room. If the server isn't there you get something like "Cannot connect to server". After a successful Connect the title bar switches to "Chat Room".

7.2 Main Chat Room

I used ImGui's `Columns` for a two-column layout. Left side is "Current Users" — the list from the server's `clients/` messages, and next to each name there's a "chat" button to open a DM with that person. Right side is the public chat: scrollable messages and a multiline box with "Send". Each line is shown as *username: content*; my own messages are right-aligned. When you send, the client pushes `public/`*username*`:` *content* to the server. When a `public/` message comes in it goes on the list and if it's from someone else it plays a sound (I put `music2.mp3` in the exe folder for that).

7.3 Direct Message Windows

Each private chat is its own window like "Private Chat with Alice". You get it by clicking "chat" next to their name. Inside you see just that conversation, an input box, and "Send" / "Close". Sending is `private/`*target*`:` *content*. When a `private/` message arrives it goes into the right `WindowInfo` and plays a different sound (`music1.mp3`) so you can tell it from the main room. If the other person leaves, the title gets "(offline)" and Send is greyed out until they're back.

7.4 Screenshots

8 Conclusion (Part 2)

I think the chat room covers what was asked: client and server with WinSock, several clients at once, connect/disconnect, broadcast and DMs, and the ImGui layout with the list on one side and the main chat in the middle plus separate windows for private chats. The two different sounds for public vs direct messages are there as well. The protocol is just strings with a

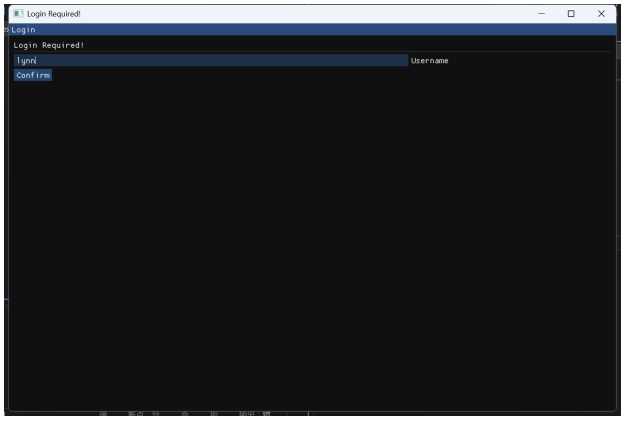

Figure 3: *Login screen with username input and Confirm buttons.*

Figure 4: *Main chat room — connections on the left, chat messages in the middle*

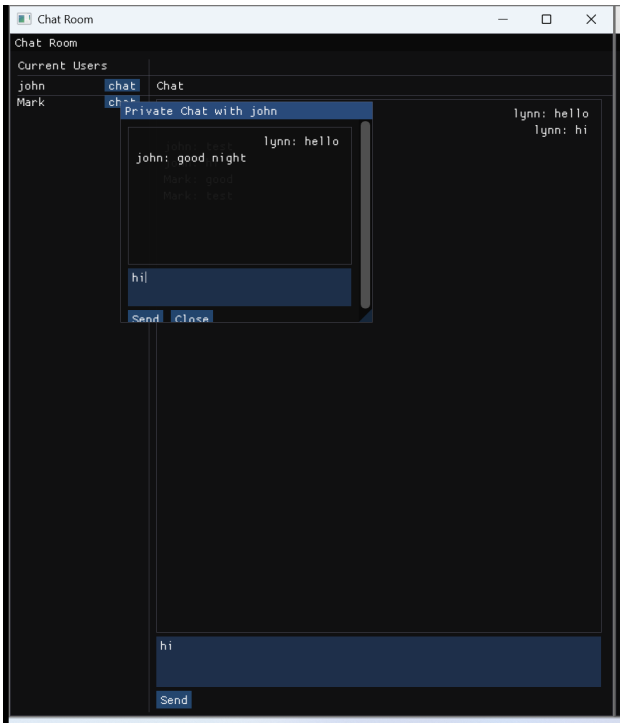

Figure 5: *A separate window for direct messages between two participants*

type prefix (`clients/`, `public/`, `private/`) and the server doesn't store any of it.

If I had more time I'd look at handling very long messages (or binary), maybe checking `send()` return values on the server, and optionally saving chat history or adding proper accounts.